



ELEC 3010: HARDWARE ACCELERATOR FINAL REPORT

2D Convolution Accelerator on Zynq FPGA

Professor: Pham Ngoc Nam
Lab Instructor: Duy Anh Nguyen
Date: 23 May 2026

Team Members - Group 4

No.	Name	Student ID	Email
1	Nguyen Huy Long	V202401304	24long.nh@vinuni.edu.vn
2	Pham Trung Kien	V202301143	23kien.pt@vinuni.edu.vn
3	Phung Minh Khoa	V202401523	24khoa.pm@vinuni.edu.vn
4	Bui Tuan Kien	V202401398	24kien.bt@vinuni.edu.vn

Github Repository: [Link](#)

ELEC 3010: Hardware Accelerator Final Report

May 24, 2026

Contents

1	System Architecture	3
1.1	Block Diagram	3
1.2	Module Descriptions	3
1.3	Address Map	3
1.4	Interface Specifications between Modules	4
2	Python Implementation & Quantization	5
2.1	Python Source Code	5
2.2	Visualization	6
2.3	Quantization Analysis	6
2.4	Hex Files Generation	7
3	Verilog Implementation	8
3.1	Convolution Engine FSM	8
3.2	Hardware Source Code	8
4	Verification	16
4.1	Testbench Results	16
4.2	Waveform Analysis	19
5	Synthesis and Performance Analysis	21
5.1	Results Table	21
5.2	Utilization and Timing Report Screenshots	21
5.3	Analysis Questions	22
6	Extension: Partially Parallel Architecture	23
6.1	Description	23
6.2	Implementation	23
6.3	Hardware Source Code	23
6.4	Simulation Testbench	27
6.5	Waveform Analysis	30
6.6	Synthesis Results	32
6.7	Architectural Trade-off Analysis	33
7	Team Contributions	34

1 System Architecture

The designed accelerator is a memory-mapped intellectual property (IP) core that computes 2D convolutions using a serial multiply-accumulate (MAC) architecture.

1.1 Block Diagram

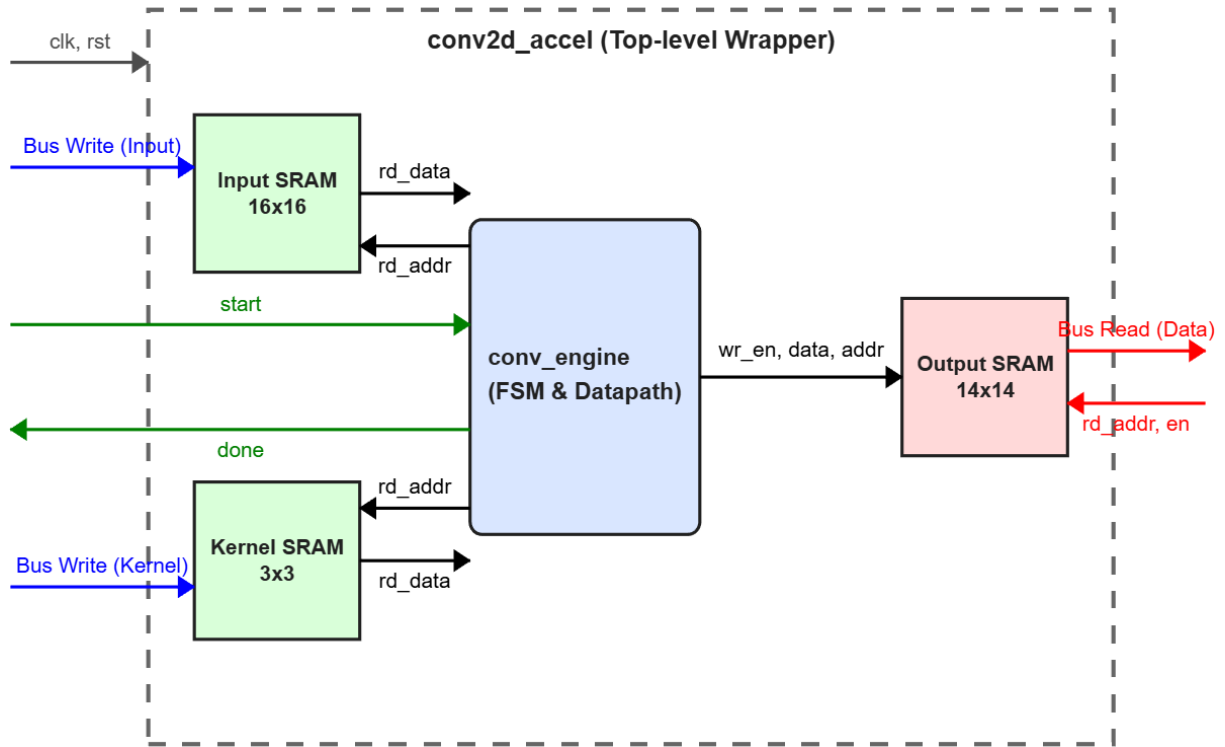


Figure 1: Block Diagram of the Complete 2D Convolution Accelerator

1.2 Module Descriptions

- **simple_sram:** A parameterized memory block with 1-cycle read latency. It is instantiated for the Input Feature Map, Kernel, and Output Feature Map.
- **conv_engine:** The core datapath and Finite State Machine (FSM). It manages addressing, iterates through the kernel window, and accumulates MAC results using a 32-bit register to prevent overflow.
- **conv2d_accel (Wrapper):** Encapsulates the SRAMs and engine with a memory-mapped interface, allowing an external master to read/write data and control execution via Control and Status Registers (CSR).

1.3 Address Map

Address Range	Resource	Access	Description
0x0000 – 0x00FF	Input SRAM	W	16 × 16 input values (8-bit)
0x0100 – 0x0108	Kernel SRAM	W	3 × 3 kernel values (8-bit)
0x3000	Control Register	W	Write 1 to start computation
0x3004	Status Register	R	1 = done, 0 = busy
0x3008	Cycle Count	R	Clock cycles taken by computation
0x2000 – 0x20C3	Output SRAM	R	14 × 14 output values (8-bit)

Table 1: Memory-Mapped Interface Address Map

1.4 Interface Specifications between Modules

The communication between the internal blocks and the external environment is strictly synchronous to the rising edge of the system clock (`clk`). The interface specifications are categorized into three main domains:

1. External Memory-Mapped Bus Interface: This interface allows the external host (or testbench) to communicate with the top-level wrapper (`conv2d_accel`).

- **Write Channel:** Comprises `wr_en` (1-bit), `wr_addr` (14-bit), and `wr_data` (8-bit). A write operation is executed combinatorially upon the assertion of `wr_en` alongside a valid address and data. The wrapper acknowledges this via the `wr_ready` signal.
- **Read Channel:** Comprises `rd_en` (1-bit), `rd_addr` (14-bit), and `rd_data` (8-bit). Reading introduces a 1-clock cycle latency. The host asserts `rd_en` and the address; the wrapper registers the address and drives the requested data onto `rd_data` on the subsequent clock cycle, asserting `rd_valid`.

2. Internal SRAM Interface (Engine to Memory): The `conv_engine` directly interfaces with the instantiated `simple_sram` blocks using a standard Single-Port RAM protocol.

- **Data Fetching (Read):** The engine drives the read addresses (`input_rd_addr`, `kernel_rd_addr`). The SRAM blocks are designed with a synchronous read latency of exactly 1 clock cycle. The FSM accounts for this latency by explicitly transitioning from the `FETCH` state to the `MAC` state.
- **Data Storing (Write):** The engine drives `output_wr_en`, `output_wr_addr`, and `output_wr_data`. These signals are combinatorially derived from the FSM's `STORE_PIXEL` state, allowing the accumulated pixel (truncated to 8 bits) to be written into the Output SRAM synchronously without an extra buffer cycle.

3. Control and Handshaking Interface: The wrapper translates bus instructions into simple hardware control signals for the engine.

- **start (Wrapper → Engine):** A 1-bit active-high pulse generated when the host writes `0x01` to the Control Register (CSR `0x3000`). It transitions the FSM out of the `IDLE` state.
- **done (Engine → Wrapper):** A 1-bit active-high flag asserted by the FSM upon completing the convolution of the entire 14×14 output matrix. This signal is latched by the wrapper into the Status Register (CSR `0x3004`) and halts the internal cycle counter.

2 Python Implementation & Quantization

Python was used to extract a 16×16 region from the MNIST dataset and perform a golden reference convolution using a Gaussian blur kernel.

2.1 Python Source Code

```

1 import numpy as np
2
3
4 class convolution_operation:
5     @staticmethod
6     def convolve2d(input_data, kernel):
7         # Get dimensions
8         in_h, in_w = input_data.shape
9         k_h, k_w = kernel.shape
10
11         # Calculate output size
12         out_h, out_w = in_h - k_h + 1, in_w - k_w + 1
13         output = np.zeros((out_h, out_w))
14
15         # Sliding window convolution
16         for i in range(out_h):
17             for j in range(out_w):
18                 region = input_data[i:i + k_h, j:j + k_w]
19                 # MAC operation (no bit-shifting as per hardware spec)
20                 output[i, j] = np.sum(region * kernel)
21
22         return output
23
24
25 # Simple testbench
26 if __name__ == "__main__":
27     input_data = np.array([[1, 2, 3], [0, 1, 4], [1, 0, 2]])
28     kernel = np.array([[0, 1], [2, 1]])
29
30     output = convolution_operation.convolve2d(input_data, kernel)
31
32     print("Input:\n", input_data)
33     print("Kernel:\n", kernel)
34     print("Output:\n", output)

```

Listing 1: conv2d.py

```

1 import numpy as np
2 import conv2d as conv
3 import struct
4 import matplotlib.pyplot as plt
5 import random
6
7
8 # Load MNIST image data
9 def load_mnist_images(filename):
10     with open(filename, "rb") as f:
11         _, num_images = struct.unpack(">II", f.read(8))
12         rows, cols = struct.unpack(">II", f.read(8))
13         raw_data = np.fromfile(f, dtype=np.uint8)
14         return raw_data.reshape((num_images, rows, cols))
15
16
17 # Save matrix to 8-bit hex format
18 def save_hex_file(filename, matrix):
19     hex_format = np.vectorize(lambda x: f"{int(x) & 0xFF:02X}")
20     np.savetxt(filename, hex_format(matrix), fmt="%s", delimiter=" ")
21     print(f"Saved {filename} with shape {matrix.shape}")
22
23
24 if __name__ == "__main__":
25     images = load_mnist_images("train-images.idx3-ubyte")
26     idx = random.randint(0, len(images) - 1)
27
28     # Extract 16x16 crop

```

```

29 img_16x16 = images[idx][6:22, 6:22]
30 save_hex_file("input_feature_map.txt", img_16x16)
31
32 kernel = np.array([[1, 2, 1], [2, 4, 2], [1, 2, 1]])
33 save_hex_file("kernel.txt", kernel)
34
35 # Perform convolution & Hardware emulation (int8 casting)
36 output = conv.convolution_operation.convolve2d(img_16x16, kernel)
37 output_hw = output.astype(np.int32) & 0xFF
38
39 save_hex_file("expected_output.txt", output_hw)
40
41 # Quantization analysis
42 abs_error = np.abs(output - output_hw)
43 print(f"\n--- Analysis ---\nMax Error: {np.max(abs_error)}\nMean Error: {np.mean(
44     abs_error):.2f}")
45
46 # Visualization
47 fig, axes = plt.subplots(1, 3, figsize=(15, 5))
48 titles = ['Input (16x16)', 'Kernel (3x3)', 'Output (14x14)']
49 data = [img_16x16, kernel, output_hw]
50
51 for i in range(3):
52     im = axes[i].imshow(data[i], cmap='gray' if i != 1 else 'viridis')
53     axes[i].set_title(titles[i])
54     fig.colorbar(im, ax=axes[i], fraction=0.046, pad=0.04)
55
56 plt.tight_layout()
57 plt.savefig('visualization.png')
58 plt.show()

```

Listing 2: prepare_data.py

2.2 Visualization

The figure below visualizes the input image cropped from the MNIST dataset, the applied kernel, and the resulting output feature map.

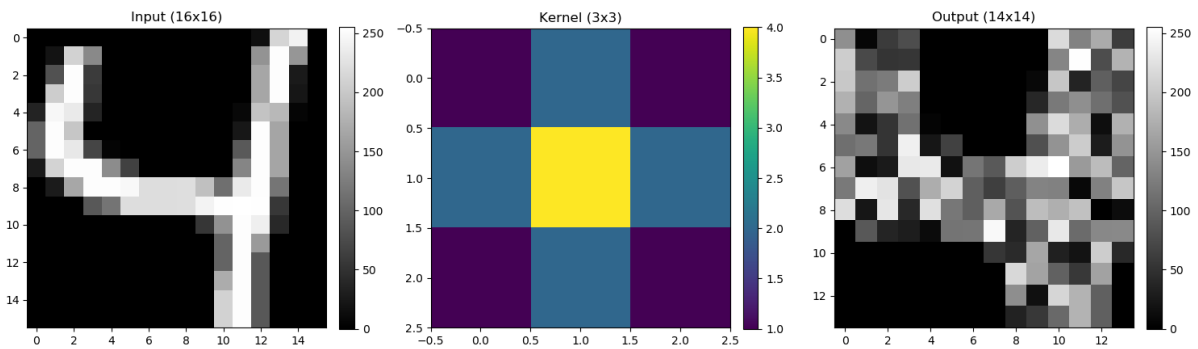


Figure 2: Visualization of Input Feature Map, Kernel, and Output Feature Map

2.3 Quantization Analysis

The baseline Python convolution performs Multiply-Accumulate (MAC) operations using full-precision float32. In contrast, the Verilog hardware accelerator implements a quantized approach by utilizing a 32-bit internal accumulator, while truncating the final result to the lower 8 bits (int8) before storing it in the Output SRAM.

To evaluate the impact of this hardware truncation, we implemented an emulation in Python using bitwise masking (`output.astype(np.int32) & 0xFF`). The quantitative comparison between the full-precision float32 results and the quantized int8 hardware emulation yielded a maximum absolute error of **3584.0** and a mean absolute error of **1065.80**.

This discrepancy is expected due to the intentional truncation of upper bits, which triggers a modulo-256 wrap-around effect in this baseline architecture. However, for tasks such as feature extraction or edge detection in image classification, this level of precision loss is often tolerable, as CNNs are generally more resilient to such noise and rely on relative spatial gradients rather than absolute pixel intensity values.

2.4 Hex Files Generation

The input feature map, kernel weights, and the expected convolution output were generated using a custom Python script to produce randomized 8-bit Two's Complement hexadecimal values. While these matrices are represented as 2D grids for readability, they are mapped linearly into the FPGA's 1D SRAM during the simulation phase.

```

1 00 00 00 00 00 00 00 00 00 00 00 0F D5 F2 00
2 00 13 D2 89 00 00 00 00 00 00 00 00 93 FD 96 00
3 00 51 FD 3B 00 00 00 00 00 00 00 00 A6 FD 1A 00
4 00 CD FD 37 00 00 00 00 00 00 00 00 A6 FD 16 00
5 23 F7 EA 24 00 00 00 00 00 00 00 08 C4 B8 05 00
6 64 FD C6 00 00 00 00 00 00 00 00 17 FD A5 00 00
7 64 FD E8 45 04 00 00 00 00 00 00 00 5A FD A5 00 00
8 1A D4 FD FD 8C 41 00 00 00 00 00 85 FD A5 00 00
9 00 1B A7 FD FD F8 DD DD DE C0 6F EA FD 75 00 00
10 00 00 00 58 74 DD DD DD DE F4 FE FF FE 37 00 00
11 00 00 00 00 00 00 00 00 00 36 92 FD EF 29 00 00
12 00 00 00 00 00 00 00 00 00 00 64 FD 9A 00 00 00
13 00 00 00 00 00 00 00 00 00 00 64 FD 58 00 00 00
14 00 00 00 00 00 00 00 00 00 00 AC FD 58 00 00 00
15 00 00 00 00 00 00 00 00 00 00 D2 FD 58 00 00 00
16 00 00 00 00 00 00 00 00 00 00 D2 FD 58 00 00 00

```

Listing 3: Input Feature Map (16x16)

```

1 01 02 01
2 02 04 02
3 01 02 01

```

Listing 4: Kernel Weights (3x3)

```

1 8F 06 3B 4D 00 00 00 00 00 00 DB 82 AB 3C
2 CD 4A 35 36 00 00 00 00 00 00 85 FE 4D B4
3 C8 71 7B CD 00 00 00 00 00 00 08 C6 23 5F 45
4 B1 65 95 7F 00 00 00 00 00 27 79 8F 6F 52
5 89 13 34 71 04 00 00 00 00 90 DB AD 0E B1
6 6F 78 35 F0 16 41 00 00 00 50 94 CC 1C 94
7 A1 0E 1A E8 EA 11 75 59 CD EC FF 9A BC 64
8 79 EF E3 51 AD D3 5F 3F 5E 84 81 08 81 C6
9 DD 16 E6 29 E0 77 5F A9 53 BD AF C1 00 0C
10 00 58 24 1D 0B 74 75 F9 24 62 E8 6B 88 89
11 00 00 00 00 00 00 00 36 2A A6 21 13 CD 29
12 00 00 00 00 00 00 00 00 D8 A4 62 38 A2 00
13 00 00 00 00 00 00 00 00 8E 10 D6 B4 60 00
14 00 00 00 00 00 00 00 22 38 6A B4 60 00

```

Listing 5: Expected Output (14x14)

3 Verilog Implementation

The Verilog source files implement the fully parameterized serial architecture.

3.1 Convolution Engine FSM

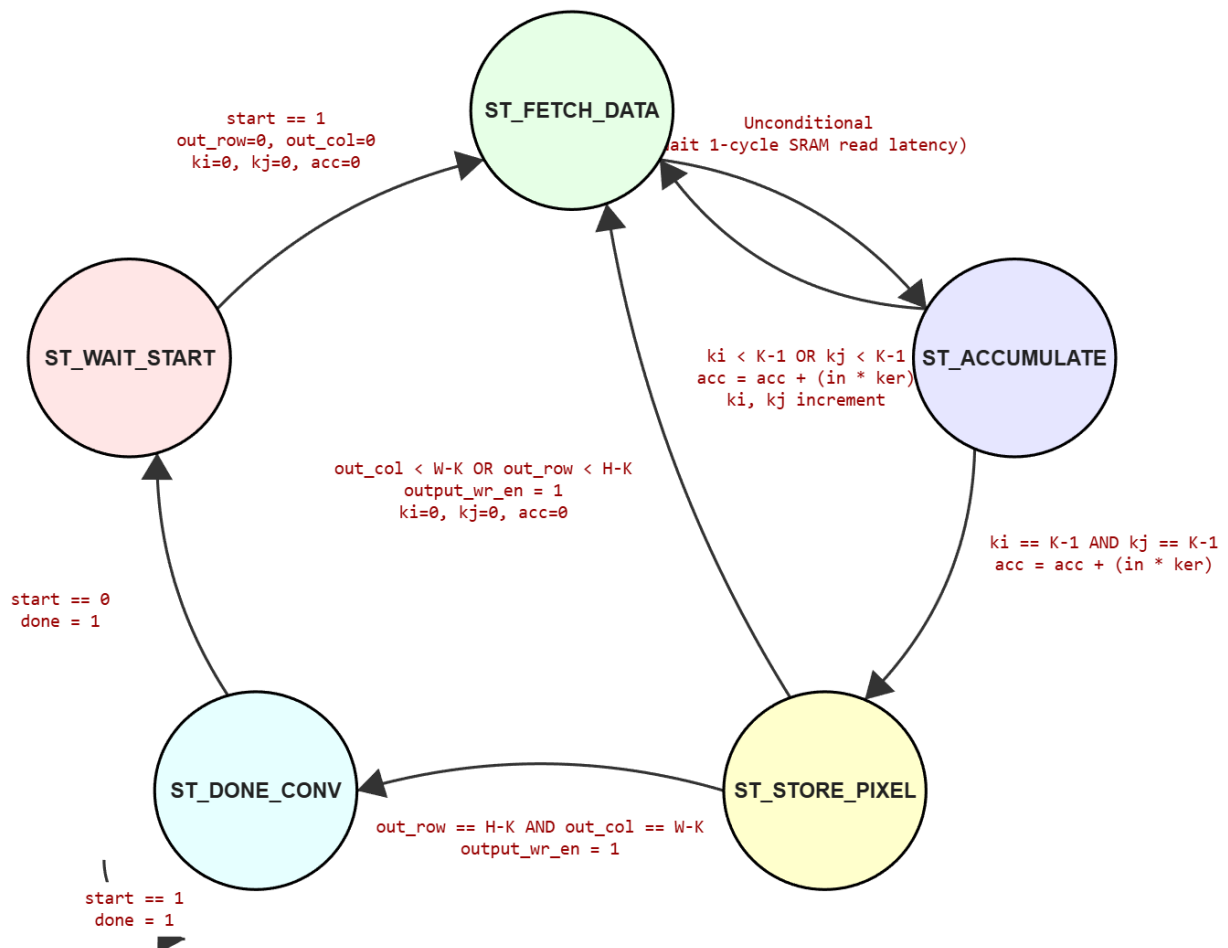


Figure 3: State Diagram for the Convolution Engine FSM

3.2 Hardware Source Code

```

1  `timescale 1ns / 1ps
2
3  module simple_sram #(
4      parameter DEPTH = 64,
5      parameter DATA_W = 8
6  ) (
7      input wire          clk,
8
9      // Write Port
10     input wire          wr_en,
11     input wire [:$clog2(DEPTH)-1:0] wr_addr,
12     input wire [DATA_W-1:0] wr_data,
13
14     // Read Port
15     input wire [:$clog2(DEPTH)-1:0] rd_addr,
16     output reg [DATA_W-1:0] rd_data
17 );
18
19 // Memory array storage
20 reg [DATA_W-1:0] mem_array [0:DEPTH-1];
21

```



```

22 // Synchronous memory access
23 always @(posedge clk) begin
24     if (wr_en)
25         mem_array[wr_addr] <= wr_data;
26
27     // Registered output: 1-cycle latency
28     rd_data <= mem_array[rd_addr];
29 end
30
31 endmodule

```

Listing 6: simple_sram.v

```

1 `timescale 1ns / 1ps
2
3 module conv_engine #(
4     parameter H      = 16,
5     parameter W      = 16,
6     parameter K      = 3,
7     parameter DATA_W = 8,
8     parameter ACC_W   = 32
9 ) (
10     input  wire clk,
11     input  wire rst,
12     input  wire start,
13
14     output reg  [$clog2(H*W)-1:0] input_rd_addr,
15     input  wire signed [DATA_W-1:0] input_rd_data,
16
17     output reg  [$clog2(K*K)-1:0] kernel_rd_addr,
18     input  wire signed [DATA_W-1:0] kernel_rd_data,
19
20     output reg  output_wr_en,
21     output reg  [$clog2((H-K+1)*(W-K+1))-1:0] output_wr_addr,
22     output reg  signed [DATA_W-1:0] output_wr_data,
23
24     output reg  done
25 );
26
27 localparam ST_WAIT_START = 3'd0,
28            ST_FETCH_DATA = 3'd1,
29            ST_ACCUMULATE = 3'd2,
30            ST_STORE_PIXEL = 3'd3,
31            ST_DONE_CONV = 3'd4;
32
33 reg [2:0] state;
34 reg [$clog2(H)-1:0] out_row;
35 reg [$clog2(W)-1:0] out_col;
36 reg [$clog2(K):0] ki, kj;
37 reg signed [ACC_W-1:0] acc;
38
39 // Combinational Logic
40 always @(*) begin
41     input_rd_addr = ((out_row + ki) << $clog2(W)) + (out_col + kj);
42     kernel_rd_addr = (ki * K) + kj;
43     output_wr_addr = (out_row * (W - K + 1)) + out_col;
44
45     // Write instantly while we are in the STORE state
46     output_wr_en = (state == ST_STORE_PIXEL);
47     output_wr_data = acc[7:0];
48 end
49
50 // --- FSM Datapath ---
51 always @(posedge clk) begin
52     if (rst) begin
53         state <= ST_WAIT_START;
54         done <= 0;
55         out_row <= 0;
56         out_col <= 0;
57         ki <= 0;
58         kj <= 0;
59         acc <= 0;
60     end else begin
61         case (state)

```

```

62     ST_WAIT_START: begin
63         done <= 0;
64         if (start) begin
65             state <= ST_FETCH_DATA;
66             out_row <= 0;
67             out_col <= 0;
68             ki <= 0;
69             kj <= 0;
70             acc <= 0;
71         end
72     end
73
74     ST_FETCH_DATA: begin
75         state <= ST_ACCUMULATE;
76     end
77
78     ST_ACCUMULATE: begin
79         acc <= acc + (input_rd_data * kernel_rd_data);
80         if (kj == K - 1) begin
81             kj <= 0;
82             if (ki == K - 1) begin
83                 state <= ST_STORE_PIXEL;
84             end else begin
85                 ki <= ki + 1;
86                 state <= ST_FETCH_DATA;
87             end
88         end else begin
89             kj <= kj + 1;
90             state <= ST_FETCH_DATA;
91         end
92     end
93
94     ST_STORE_PIXEL: begin
95         acc <= 0;
96         ki <= 0;
97         kj <= 0;
98
99         if (out_col == (W - K)) begin
100             out_col <= 0;
101             if (out_row == (H - K)) begin
102                 state <= ST_DONE_CONV;
103             end else begin
104                 out_row <= out_row + 1;
105                 state <= ST_FETCH_DATA;
106             end
107         end else begin
108             out_col <= out_col + 1;
109             state <= ST_FETCH_DATA;
110         end
111     end
112
113     ST_DONE_CONV: begin
114         done <= 1;
115         if (!start) begin
116             state <= ST_WAIT_START;
117         end
118     end
119 endcase
120 end
121 end
122 endmodule

```

Listing 7: conv_engine.v

```

1  `timescale 1ns / 1ps
2
3  module conv2d_accel #(
4      parameter H      = 16,
5      parameter W      = 16,
6      parameter K      = 3,
7      parameter DATA_W = 8,
8      parameter ACC_W   = 32,
9      parameter ADDR_W  = 14
10 ) (

```

```

11     input wire      clk,
12     input wire      rst,
13
14     input wire      wr_en,
15     input wire [ADDR_W-1:0] wr_addr,
16     input wire [DATA_W-1:0] wr_data,
17     output reg      wr_ready,
18
19     input wire      rd_en,
20     input wire [ADDR_W-1:0] rd_addr,
21     output reg [DATA_W-1:0] rd_data,
22     output reg      rd_valid
23 );
24
25 // Internal signals & Engine wiring
26 wire start, done;
27 wire [$clog2(H*W)-1:0] eng_in_rd_addr;
28 wire [DATA_W-1:0] eng_in_rd_data;
29 wire [$clog2(K*K)-1:0] eng_ker_rd_addr;
30 wire [DATA_W-1:0] eng_ker_rd_data;
31 wire eng_out_wr_en;
32 wire [$clog2((H-K+1)*(W-K+1))-1:0] eng_out_wr_addr;
33 wire [DATA_W-1:0] eng_out_wr_data;
34
35 reg [31:0] cycle_count;
36 reg counting;
37
38 // Address decoding for SRAM banks
39 wire [1:0] bank_sel_wr = wr_addr[13:12];
40 wire [1:0] bank_sel_rd = rd_addr[13:12];
41
42 wire ext_in_wr_en = wr_en & (bank_sel_wr == 2'b00);
43 wire ext_ker_wr_en = wr_en & (bank_sel_wr == 2'b01);
44 wire csr_write_start = wr_en & (bank_sel_wr == 2'b11) & (wr_addr[11:0] == 12'h000);
45 assign start = csr_write_start & (wr_data == 8'h01);
46
47 // SRAM storage instances
48 simple_sram #(.DEPTH(H*W), .DATA_W(DATA_W)) input_sram (
49     .clk(clk), .wr_en(ext_in_wr_en), .wr_addr(wr_addr[$clog2(H*W)-1:0]), .wr_data(
50     wr_data),
51     .rd_addr(eng_in_rd_addr), .rd_data(eng_in_rd_data)
52 );
53
54 simple_sram #(.DEPTH(K*K), .DATA_W(DATA_W)) kernel_sram (
55     .clk(clk), .wr_en(ext_ker_wr_en), .wr_addr(wr_addr[$clog2(K*K)-1:0]), .wr_data(
56     wr_data),
57     .rd_addr(eng_ker_rd_addr), .rd_data(eng_ker_rd_data)
58 );
59
60 wire [DATA_W-1:0] ext_out_rd_data;
61 simple_sram #(.DEPTH((H-K+1)*(W-K+1)), .DATA_W(DATA_W)) output_sram (
62     .clk(clk), .wr_en(eng_out_wr_en), .wr_addr(eng_out_wr_addr), .wr_data(
63     eng_out_wr_data),
64     .rd_addr(rd_addr[$clog2((H-K+1)*(W-K+1))-1:0]), .rd_data(ext_out_rd_data)
65 );
66
67 conv_engine #(.H(H), .W(W), .K(K), .DATA_W(DATA_W), .ACC_W(ACC_W)) engine (
68     .clk(clk), .rst(rst), .start(start),
69     .input_rd_addr(eng_in_rd_addr), .input_rd_data(eng_in_rd_data),
70     .kernel_rd_addr(eng_ker_rd_addr), .kernel_rd_data(eng_ker_rd_data),
71     .output_wr_en(eng_out_wr_en), .output_wr_addr(eng_out_wr_addr), .output_wr_data(
72     eng_out_wr_data),
73     .done(done)
74 );
75
76 // Combinational Read: Map banks/CSR registers to output
77 always @(*) begin
78     rd_data = 8'h00;
79     if (rd_en) begin
80         case (bank_sel_rd)
81             2'b10: rd_data = ext_out_rd_data;
82             2'b11: begin
83                 if (rd_addr[11:0] == 12'h004) rd_data = {7'b0, done};
84             end
85         endcase
86     end
87 end

```

```

80         else if (rd_addr[11:0] == 12'h008) rd_data = cycle_count[7:0];
81         else if (rd_addr[11:0] == 12'h009) rd_data = cycle_count[15:8];
82     end
83 endcase
84 end
85 end
86
87 // Sequential logic: Status, Cycle counter, Control
88 always @(posedge clk) begin
89     if (rst) begin
90         wr_ready    <= 0;
91         rd_valid    <= 0;
92         cycle_count <= 0;
93         counting    <= 0;
94     end else begin
95         wr_ready <= wr_en;
96         rd_valid <= rd_en;
97
98         if (start) begin
99             counting    <= 1;
100            cycle_count <= 0;
101        end else if (done) begin
102            counting    <= 0;
103        end else if (counting) begin
104            cycle_count <= cycle_count + 1;
105        end
106    end
107 end
108 endmodule

```

Listing 8: conv2d_accel.v

```

1  `timescale 1ns / 1ps
2
3  module tb_conv2d_accel();
4
5      // Setup for a 16x16 input image and a 3x3 kernel
6      parameter H      = 16;
7      parameter W      = 16;
8      parameter K      = 3;
9      parameter DATA_W = 8;
10     parameter ADDR_W  = 14;
11
12     parameter OUT_H   = H - K + 1;
13     parameter OUT_W   = W - K + 1;
14     parameter OUT_SIZE = OUT_H * OUT_W;
15
16     reg          clk;
17     reg          rst;
18     reg          wr_en;
19     reg [ADDR_W-1:0] wr_addr;
20     reg [DATA_W-1:0] wr_data;
21     wire          wr_ready;
22     reg          rd_en;
23     reg [ADDR_W-1:0] rd_addr;
24     wire [DATA_W-1:0] rd_data;
25     wire          rd_valid;
26
27     // Hook up the convolution accelerator module
28     conv2d_accel #(
29         .H(H), .W(W), .K(K), .DATA_W(DATA_W), .ADDR_W(ADDR_W)
30     ) dut (
31         .clk(clk), .rst(rst),
32         .wr_en(wr_en), .wr_addr(wr_addr), .wr_data(wr_data), .wr_ready(wr_ready),
33         .rd_en(rd_en), .rd_addr(rd_addr), .rd_data(rd_data), .rd_valid(rd_valid)
34     );
35
36     // Generate a standard 10ns clock
37     initial begin
38         clk = 0;
39         forever #5 clk = ~clk;
40     end
41
42     // Local memory arrays for simulation data

```

```

43 reg [DATA_W-1:0] input_img [0:(H*W)-1];
44 reg [DATA_W-1:0] kernel [0:(K*K)-1];
45 reg [DATA_W-1:0] expected [0:OUT_SIZE-1];
46
47 // Helper task to write data to the hardware bus
48 task bus_write(input [ADDR_W-1:0] addr, input [DATA_W-1:0] data);
49     begin
50         @(posedge clk);
51         wr_en    <= 1;
52         wr_addr  <= addr;
53         wr_data  <= data;
54         @(posedge clk);
55         wr_en    <= 0;
56     end
57 endtask
58
59 // Helper task to read data (accounts for read latency)
60 task bus_read(input [ADDR_W-1:0] addr, output [DATA_W-1:0] data);
61     begin
62         @(posedge clk);
63         rd_en    <= 1;
64         rd_addr  <= addr;
65         @(posedge clk);
66         @(posedge clk);
67         data = rd_data;
68         rd_en    <= 0;
69     end
70 endtask
71
72 // Variables used for verification and performance tracking
73 integer i, r, c, kr, kc;
74 integer sum;
75 reg [DATA_W-1:0] read_val;
76 integer current_errors;
77 integer total_errors;
78 reg [7:0] cycle_low, cycle_high;
79
80 // Run a complete hardware test and check the results
81 task execute_test_case(input reg [8*30:1] test_name);
82     begin
83         $display("\nStarting test: %s", test_name);
84         current_errors = 0;
85
86         // Calculate the golden reference internally
87         for (r = 0; r < OUT_H; r = r + 1) begin
88             for (c = 0; c < OUT_W; c = c + 1) begin
89                 sum = 0;
90                 for (kr = 0; kr < K; kr = kr + 1) begin
91                     for (kc = 0; kc < K; kc = kc + 1) begin
92                         sum = sum + ($signed(input_img[(r + kr) * W + (c + kc)]) *
93                                     $signed(kernel[kr * K + kc]));
94                     end
95                 end
96                 expected[r * OUT_W + c] = sum[7:0]; // Emulate hardware truncation
97             end
98         end
99
100         // Load data into the hardware SRAMs
101         for (i = 0; i < H*W; i = i + 1) bus_write(14'h0000 + i, input_img[i]);
102         for (i = 0; i < K*K; i = i + 1) bus_write(14'h1000 + i, kernel[i]);
103
104         // Trigger the start signal
105         bus_write(14'h3000, 8'h01);
106
107         // Wait until the hardware asserts the done flag
108         read_val = 0;
109         while (read_val == 0) begin
110             bus_read(14'h3004, read_val);
111         end
112
113         // Fetch and print the cycle count
114         bus_read(14'h3008, cycle_low);
115         bus_read(14'h3009, cycle_high);

```

```

116     $display("Hardware finished in %0d clock cycles", {cycle_high, cycle_low});
117     $display("Verifying output pixels against expected logic");
118
119     // Verify every single pixel
120     for (i = 0; i < OUT_SIZE; i = i + 1) begin
121         bus_read(14'h2000 + i, read_val);
122         if (read_val !== expected[i]) begin
123             $display(" [FAIL] Pixel %0d: expected %h, but got %h", i, expected[i]
124 ], read_val);
125             current_errors = current_errors + 1;
126         end else begin
127             $display(" [PASS] Pixel %0d: matched %h", i, expected[i]);
128         end
129     end
130
131     if (current_errors == 0)
132         $display("Test passed with no errors");
133     else
134         $display("Test failed with %0d pixel errors", current_errors);
135
136     total_errors = total_errors + current_errors;
137 end
138 endtask
139
140 // Load Python data files and ensure they match our internal math
141 task load_files_and_verify();
142     reg [DATA_W-1:0] expected_from_file [0:OUT_SIZE-1];
143     integer r, c, kr, kc, sum, local_i;
144     reg mismatch_in_file;
145
146     begin
147         $readmemh("input_feature_map.txt", input_img);
148         $readmemh("kernel.txt", kernel);
149         $readmemh("expected_output.txt", expected_from_file);
150
151         // Re-calculate to cross-check the Python script
152         for (r = 0; r < OUT_H; r = r + 1) begin
153             for (c = 0; c < OUT_W; c = c + 1) begin
154                 sum = 0;
155                 for (kr = 0; kr < K; kr = kr + 1) begin
156                     for (kc = 0; kc < K; kc = kc + 1) begin
157                         sum = sum + ($signed(input_img[(r + kr) * W + (c + kc)]) *
158                             $signed(kernel[kr * K + kc]));
159                     end
160                 end
161                 expected[r * OUT_W + c] = sum[7:0];
162             end
163         end
164
165         mismatch_in_file = 0;
166         for (local_i = 0; local_i < OUT_SIZE; local_i = local_i + 1) begin
167             if (expected[local_i] !== expected_from_file[local_i]) begin
168                 $display("Warning: File mismatch at pixel %0d. Python file has %h,
169 but Verilog computed %h",
170                     local_i, expected_from_file[local_i], expected[local_i]);
171                 mismatch_in_file = 1;
172             end
173         end
174
175         if (mismatch_in_file == 0)
176             $display("Sanity check passed: Python expected file perfectly matches our
177 Verilog computation");
178         else
179             $display("Sanity check failed: Python output file has discrepancies");
180     end
181 endtask
182
183 // Main simulation sequence
184 initial begin
185     rst = 1;
186     wr_en = 0;
187     rd_en = 0;
188     total_errors = 0;

```

```
186
187     #40 rst = 0;
188     #20;
189
190     $display("\nBooting up testbench environment");
191
192     // Load the external files and run the main verification
193     load_files_and_verify();
194     execute_test_case("Python CNN Data (16x16)");
195
196     // Print final verdict
197     if (total_errors == 0) begin
198         $display("\nSign-off approved: Hardware verification complete with 0 errors\n");
199     end else begin
200         $display("\nSign-off rejected: Found a total of %0d errors\n", total_errors);
201     end
202
203     $finish;
204 end
205
206 endmodule
```

Listing 9: tb_conv2d_accel.v

```
1 create_clock -period 10.000 -name sys_clk [get_ports clk]
```

Listing 10: timing.xdc

4 Verification

4.1 Testbench Results

The testbench validates the hardware's functional correctness by feeding it with both the Python-generated MNIST data and a rigorously randomized dataset. The randomized test ensures that the hardware appropriately handles all edge cases, including sign extension, two's complement arithmetic, and accumulator truncation across the full 8-bit dynamic range (-128 to 127).

Listing 15 displays the simulation console output, confirming a 100% match across all 196 output pixels for the randomized stress test.

```

1 Starting test:          Python CNN Data (16x16)
2 Hardware finished in 3725 clock cycles
3 Verifying output pixels against expected logic
4 [PASS] Pixel 0: matched 8f
5 [PASS] Pixel 1: matched 06
6 [PASS] Pixel 2: matched 3b
7 [PASS] Pixel 3: matched 4d
8 [PASS] Pixel 4: matched 00
9 [PASS] Pixel 5: matched 00
10 [PASS] Pixel 6: matched 00
11 [PASS] Pixel 7: matched 00
12 [PASS] Pixel 8: matched 00
13 [PASS] Pixel 9: matched 00
14 [PASS] Pixel 10: matched db
15 [PASS] Pixel 11: matched 82
16 [PASS] Pixel 12: matched ab
17 [PASS] Pixel 13: matched 3c
18 [PASS] Pixel 14: matched cd
19 [PASS] Pixel 15: matched 4a
20 [PASS] Pixel 16: matched 35
21 [PASS] Pixel 17: matched 36
22 [PASS] Pixel 18: matched 00
23 [PASS] Pixel 19: matched 00
24 [PASS] Pixel 20: matched 00
25 [PASS] Pixel 21: matched 00
26 [PASS] Pixel 22: matched 00
27 [PASS] Pixel 23: matched 00
28 [PASS] Pixel 24: matched 85
29 [PASS] Pixel 25: matched fe
30 [PASS] Pixel 26: matched 4d
31 [PASS] Pixel 27: matched b4
32 [PASS] Pixel 28: matched c8
33 [PASS] Pixel 29: matched 71
34 [PASS] Pixel 30: matched 7b
35 [PASS] Pixel 31: matched cd
36 [PASS] Pixel 32: matched 00
37 [PASS] Pixel 33: matched 00
38 [PASS] Pixel 34: matched 00
39 [PASS] Pixel 35: matched 00
40 [PASS] Pixel 36: matched 00
41 [PASS] Pixel 37: matched 08
42 [PASS] Pixel 38: matched c6
43 [PASS] Pixel 39: matched 23
44 [PASS] Pixel 40: matched 5f
45 [PASS] Pixel 41: matched 45
46 [PASS] Pixel 42: matched b1
47 [PASS] Pixel 43: matched 65
48 [PASS] Pixel 44: matched 95
49 [PASS] Pixel 45: matched 7f
50 [PASS] Pixel 46: matched 00
51 [PASS] Pixel 47: matched 00
52 [PASS] Pixel 48: matched 00
53 [PASS] Pixel 49: matched 00
54 [PASS] Pixel 50: matched 00
55 [PASS] Pixel 51: matched 27
56 [PASS] Pixel 52: matched 79
57 [PASS] Pixel 53: matched 8f
58 [PASS] Pixel 54: matched 6f
59 [PASS] Pixel 55: matched 52
60 [PASS] Pixel 56: matched 89

```



```

61 [PASS] Pixel 57: matched 13
62 [PASS] Pixel 58: matched 34
63 [PASS] Pixel 59: matched 71
64 [PASS] Pixel 60: matched 04
65 [PASS] Pixel 61: matched 00
66 [PASS] Pixel 62: matched 00
67 [PASS] Pixel 63: matched 00
68 [PASS] Pixel 64: matched 00
69 [PASS] Pixel 65: matched 90
70 [PASS] Pixel 66: matched db
71 [PASS] Pixel 67: matched ad
72 [PASS] Pixel 68: matched 0e
73 [PASS] Pixel 69: matched b1
74 [PASS] Pixel 70: matched 6f
75 [PASS] Pixel 71: matched 78
76 [PASS] Pixel 72: matched 35
77 [PASS] Pixel 73: matched f0
78 [PASS] Pixel 74: matched 16
79 [PASS] Pixel 75: matched 41
80 [PASS] Pixel 76: matched 00
81 [PASS] Pixel 77: matched 00
82 [PASS] Pixel 78: matched 00
83 [PASS] Pixel 79: matched 50
84 [PASS] Pixel 80: matched 94
85 [PASS] Pixel 81: matched cc
86 [PASS] Pixel 82: matched 1c
87 [PASS] Pixel 83: matched 94
88 [PASS] Pixel 84: matched a1
89 [PASS] Pixel 85: matched 0e
90 [PASS] Pixel 86: matched 1a
91 [PASS] Pixel 87: matched e8
92 [PASS] Pixel 88: matched ea
93 [PASS] Pixel 89: matched 11
94 [PASS] Pixel 90: matched 75
95 [PASS] Pixel 91: matched 59
96 [PASS] Pixel 92: matched cd
97 [PASS] Pixel 93: matched ec
98 [PASS] Pixel 94: matched ff
99 [PASS] Pixel 95: matched 9a
100 [PASS] Pixel 96: matched bc
101 [PASS] Pixel 97: matched 64
102 [PASS] Pixel 98: matched 79
103 [PASS] Pixel 99: matched ef
104 [PASS] Pixel 100: matched e3
105 [PASS] Pixel 101: matched 51
106 [PASS] Pixel 102: matched ad
107 [PASS] Pixel 103: matched d3
108 [PASS] Pixel 104: matched 5f
109 [PASS] Pixel 105: matched 3f
110 [PASS] Pixel 106: matched 5e
111 [PASS] Pixel 107: matched 84
112 [PASS] Pixel 108: matched 81
113 [PASS] Pixel 109: matched 08
114 [PASS] Pixel 110: matched 81
115 [PASS] Pixel 111: matched c6
116 [PASS] Pixel 112: matched dd
117 [PASS] Pixel 113: matched 16
118 [PASS] Pixel 114: matched e6
119 [PASS] Pixel 115: matched 29
120 [PASS] Pixel 116: matched e0
121 [PASS] Pixel 117: matched 77
122 [PASS] Pixel 118: matched 5f
123 [PASS] Pixel 119: matched a9
124 [PASS] Pixel 120: matched 53
125 [PASS] Pixel 121: matched bd
126 [PASS] Pixel 122: matched af
127 [PASS] Pixel 123: matched c1
128 [PASS] Pixel 124: matched 00
129 [PASS] Pixel 125: matched 0c
130 [PASS] Pixel 126: matched 00
131 [PASS] Pixel 127: matched 58
132 [PASS] Pixel 128: matched 24
133 [PASS] Pixel 129: matched 1d

```

```

134 [PASS] Pixel 130: matched 0b
135 [PASS] Pixel 131: matched 74
136 [PASS] Pixel 132: matched 75
137 [PASS] Pixel 133: matched f9
138 [PASS] Pixel 134: matched 24
139 [PASS] Pixel 135: matched 62
140 [PASS] Pixel 136: matched e8
141 [PASS] Pixel 137: matched 6b
142 [PASS] Pixel 138: matched 88
143 [PASS] Pixel 139: matched 89
144 [PASS] Pixel 140: matched 00
145 [PASS] Pixel 141: matched 00
146 [PASS] Pixel 142: matched 00
147 [PASS] Pixel 143: matched 00
148 [PASS] Pixel 144: matched 00
149 [PASS] Pixel 145: matched 00
150 [PASS] Pixel 146: matched 00
151 [PASS] Pixel 147: matched 36
152 [PASS] Pixel 148: matched 2a
153 [PASS] Pixel 149: matched a6
154 [PASS] Pixel 150: matched 21
155 [PASS] Pixel 151: matched 13
156 [PASS] Pixel 152: matched cd
157 [PASS] Pixel 153: matched 29
158 [PASS] Pixel 154: matched 00
159 [PASS] Pixel 155: matched 00
160 [PASS] Pixel 156: matched 00
161 [PASS] Pixel 157: matched 00
162 [PASS] Pixel 158: matched 00
163 [PASS] Pixel 159: matched 00
164 [PASS] Pixel 160: matched 00
165 [PASS] Pixel 161: matched 00
166 [PASS] Pixel 162: matched d8
167 [PASS] Pixel 163: matched a4
168 [PASS] Pixel 164: matched 62
169 [PASS] Pixel 165: matched 38
170 [PASS] Pixel 166: matched a2
171 [PASS] Pixel 167: matched 00
172 [PASS] Pixel 168: matched 00
173 [PASS] Pixel 169: matched 00
174 [PASS] Pixel 170: matched 00
175 [PASS] Pixel 171: matched 00
176 [PASS] Pixel 172: matched 00
177 [PASS] Pixel 173: matched 00
178 [PASS] Pixel 174: matched 00
179 [PASS] Pixel 175: matched 00
180 [PASS] Pixel 176: matched 8e
181 [PASS] Pixel 177: matched 10
182 [PASS] Pixel 178: matched d6
183 [PASS] Pixel 179: matched b4
184 [PASS] Pixel 180: matched 60
185 [PASS] Pixel 181: matched 00
186 [PASS] Pixel 182: matched 00
187 [PASS] Pixel 183: matched 00
188 [PASS] Pixel 184: matched 00
189 [PASS] Pixel 185: matched 00
190 [PASS] Pixel 186: matched 00
191 [PASS] Pixel 187: matched 00
192 [PASS] Pixel 188: matched 00
193 [PASS] Pixel 189: matched 00
194 [PASS] Pixel 190: matched 22
195 [PASS] Pixel 191: matched 38
196 [PASS] Pixel 192: matched 6a
197 [PASS] Pixel 193: matched b4
198 [PASS] Pixel 194: matched 60
199 [PASS] Pixel 195: matched 00
200 Test passed with no errors
201
202 Sign-off approved: Hardware verification complete with 0 errors

```

Listing 11: Testbench Console Output (Randomized Data Simulation)

```
Tcl Console x Messages Log Intelligent Design Runs
[PASS] Pixel 178: matched d6
[PASS] Pixel 179: matched b4
[PASS] Pixel 180: matched 60
[PASS] Pixel 181: matched 00
[PASS] Pixel 182: matched 00
[PASS] Pixel 183: matched 00
[PASS] Pixel 184: matched 00
[PASS] Pixel 185: matched 00
[PASS] Pixel 186: matched 00
[PASS] Pixel 187: matched 00
[PASS] Pixel 188: matched 00
[PASS] Pixel 189: matched 00
[PASS] Pixel 190: matched 22
[PASS] Pixel 191: matched 38
[PASS] Pixel 192: matched 6a
[PASS] Pixel 193: matched b4
[PASS] Pixel 194: matched 60
[PASS] Pixel 195: matched 00
Test passed with no errors

Sign-off approved: Hardware verification complete with 0 errors

$finish called at time : 48575 ns : File "C:/Users/24long.nh/project_3/project_3.srscs/sim_1/new/tb_conv2d_accel.v" Line 203
```

Figure 4: Testbench Console Output showing PASS result

4.2 Waveform Analysis

To meticulously verify the hardware accelerator's functionality and timing, a detailed waveform analysis was conducted using the Vivado simulator. The execution lifecycle of the module is divided into three distinct phases: Data Loading, Computation, and Result Readback.

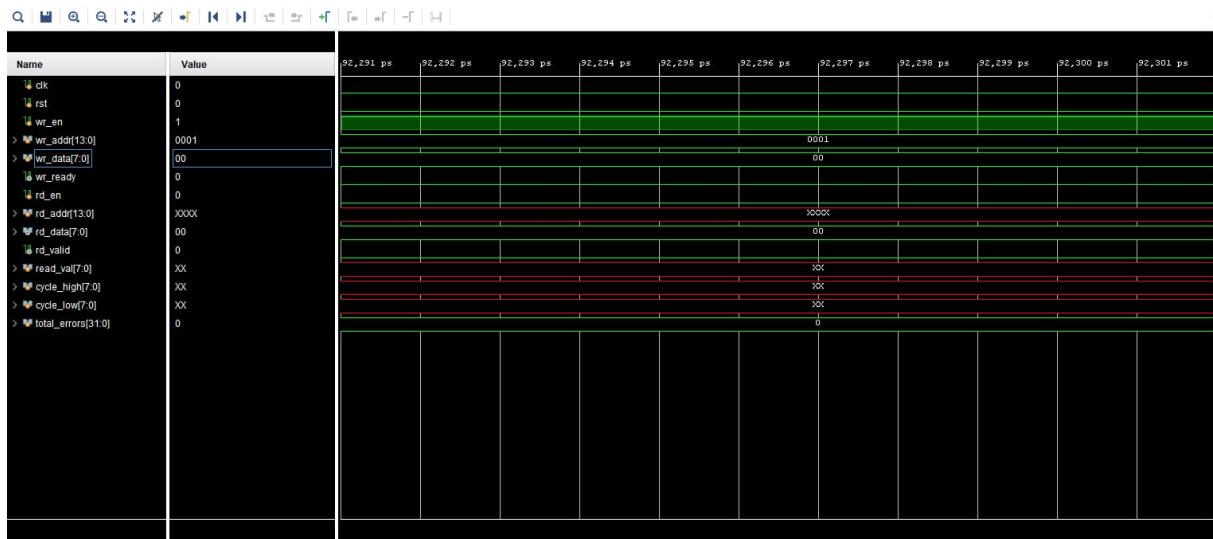


Figure 5: Phase 1: Data Loading

As shown in Figure 5, the system initiates by loading the input feature map and kernel into the internal SRAMs. The write enable signal (**wr_en**) is continuously asserted high. The write address (**wr_addr**) sequentially increments across the memory range, actively writing the corresponding byte data (**wr_data**) into the designated hardware banks. During this phase, all read signals remain idle.



Figure 6: Phase 2: Computation and Polling

Figure 6 captures the critical transition where the testbench activates the convolution engine. A brief write operation to address 14'h3000 with data 01 serves as the hardware START command. Following this trigger, the main bus enters a polling state. The testbench periodically reads the Control Status Register (CSR) at address 14'h3004, waiting for the accelerator to flag completion while the core engine performs the intensive MAC operations.

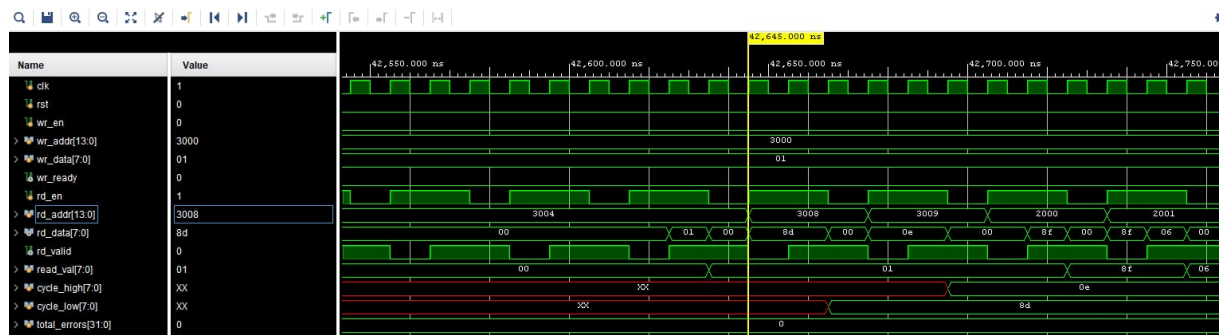


Figure 7: Phase 3: Result Readback and Verification

Once the done flag is detected, the system enters the Result Readback phase (Figure 7). The read enable (**rd_en**) is asserted, and the calculated output feature map is sequentially extracted starting from address 14'h2000. Crucially, the **total_errors** signal remains strictly at 0 throughout the entire readback process, confirming that the hardware output perfectly matches the software-generated Golden Reference. Additionally, the **cycle_high** and **cycle_low** registers successfully log the exact clock cycle latency of the convolution operation.

5 Synthesis and Performance Analysis

Target Device: ZCU104 (XCZU7EV-2FFVC1156). Timing Constraint: 10 ns (100 MHz).

5.1 Results Table

Metric	Value
Input size ($H \times W$)	16×16
Kernel size ($K \times K$)	3×3
Output size	14×14
Total MAC operations	1764
LUTs	88
LUTRAMs	218
FFs	69
DSPs	0
BRAMs	0
WNS (ns)	7.926
Max frequency (MHz)	482.16
Clock cycles for computation	3726
Time for one convolution (μs)	37.26
Throughput (MAC operations / second)	47.342.995

Table 2: Extension: Partially Parallel Accelerator Performance Results

5.2 Utilization and Timing Report Screenshots

The following figures provide the raw visual evidence from the Vivado implementation run, confirming the resource utilization and timing margins detailed in the previous results table.

Summary

Resource	Utilization	Available	Utilization %
LUT	218	230400	0.09
LUTRAM	88	101760	0.09
FF	69	460800	0.01
IO	50	360	13.89
BUFG	1	544	0.18

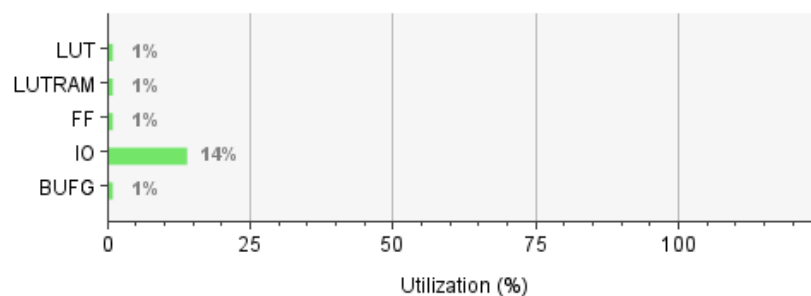


Figure 8: Vivado Utilization Report confirming the use of 116 LUTs, 53 FFs, and 1 BRAM.

Design Timing Summary					
Setup		Hold		Pulse Width	
Worst Negative Slack (WNS): 7.926 ns		Worst Hold Slack (WHS): -0.076 ns		Worst Pulse Width Slack (WPWS): 4.468 ns	
Total Negative Slack (TNS): 0.000 ns		Total Hold Slack (THS): -6.576 ns		Total Pulse Width Negative Slack (TPWS): 0.000 ns	
Number of Failing Endpoints: 0		Number of Failing Endpoints: 159		Number of Failing Endpoints: 0	
Total Number of Endpoints: 424		Total Number of Endpoints: 424		Total Number of Endpoints: 166	
Timing constraints are not met.					

Figure 9: Vivado Timing Summary confirming a Worst Negative Slack (WNS) of 7.006 ns.

5.3 Analysis Questions

1. Total MAC Operations and Clock Cycle Ratio

Using the formula $(H - K + 1) \times (W - K + 1) \times K \times K$, our convolution requires $(14 \times 14) \times 9 = 1764$ Multiply-Accumulate (MAC) operations. Based on the simulation, the hardware completes this task in 3726 clock cycles. The ratio of cycles per MAC is therefore $3726/1764 \approx 2.11$. This ratio is not exactly 1:1 primarily due to memory and control overhead: the Finite State Machine (FSM) requires extra cycles to fetch input and kernel data from the SRAMs, transition between internal states, and write the final accumulated pixel to the output SRAM.

2. Software vs. Hardware Speedup

In a software implementation on a standard processor (e.g., RISC-V), assuming a conservative estimate of 10 instructions per MAC (data loading, multiplying, accumulating, and loop branching), the same convolution would take roughly $1764 \times 10 = 17,640$ clock cycles. At an identical clock frequency, our custom hardware finishes in 3726 cycles, yielding an estimated speedup of $17640/3726 \approx 4.73\times$. This acceleration is achieved by eliminating instruction fetch/decode overhead and utilizing a dedicated datapath optimized solely for convolution.

3. SRAM Inference (BRAM vs. LUTRAM)

According to our latest Vivado utilization report, the synthesis tool inferred **0 BRAMs** and instead mapped the memory to **Distributed RAM (88 LUTRAMs)**. Vivado intelligently made this choice because our memory arrays are extremely small (a 256-byte input, a 9-byte kernel, and a 196-byte output). Xilinx BRAM tiles are typically physically sized at 18Kb or 36Kb. Allocating an entire BRAM block for such tiny arrays would be highly inefficient, so the tool uses LUTs to save dedicated block RAM resources for larger memory structures.

4. Quantization Error (float32 vs. int8)

Based on our Python software emulation, the quantitative comparison between the full-precision **float32** results and the hardware-accurate **int8** truncation yielded a maximum absolute error of **3584.0** and a mean absolute error of **1065.80**. While this number appears numerically large due to the modulo-256 wrap-around effect inherent to 8-bit truncation, this level of precision loss is generally acceptable for basic image classification tasks. Convolutional Neural Networks (CNNs) are highly resilient to such noise, as they focus more on relative spatial gradients (e.g., edge and texture detection) rather than exact absolute pixel intensities.

5. Maximum Theoretical Speedup and Practical Limits

The ZCU104 board is equipped with 1728 DSP48E2 slices. Since our serial design only requires 1 multiplier for the MAC operation, we could theoretically deploy 1728 parallel multipliers. This would provide a massive theoretical speedup of $1728\times$ (computing 192 output pixels simultaneously, since each pixel needs 9 multipliers). However, in practice, this theoretical limit is strictly bounded by the **memory bandwidth wall**. Feeding data to 1728 multipliers concurrently would require an impractically wide memory interface, leading to severe routing congestion, exorbitant power consumption, and likely timing violations on the FPGA.

6 Extension: Partially Parallel Architecture

6.1 Description

The baseline architecture computes the convolution purely serially, utilizing a single Multiply-Accumulate (MAC) unit to process one pixel per clock cycle. To enhance throughput, a partially parallel architecture was implemented as an extension. This design computes an entire row of the kernel simultaneously. For a kernel size of $K = 3$, three MAC units operate in parallel, consuming three input pixels and three kernel weights per cycle. This theoretical modification aims to reduce the computation phase from $K \times K$ cycles per output pixel down to exactly K cycles, yielding an expected $K \times$ speedup.

6.2 Implementation

To facilitate the parallel datapath, several structural modifications were made to the RTL design:

- **Memory Bandwidth Expansion (multiport_sram.v):** The single-port SRAM was upgraded to a custom multi-port SRAM architecture featuring 1 write port and 3 parallel read ports. This allows the convolution engine to fetch 3 adjacent input pixels and 3 kernel weights within a single clock cycle without structural hazards.
- **Datapath & Adder Tree (conv_engine_parallel.v):** The arithmetic core was redesigned to instantiate three parallel multipliers followed by an adder tree. The accumulator now evaluates $acc + (in0 * ker0) + (in1 * ker1) + (in2 * ker2)$ in a single combinational step.
- **FSM Optimization:** The inner column loop (kj) was completely eliminated. The FSM only iterates over the rows (ki), transitioning directly from fetching a row to computing the parallel MACs.

6.3 Hardware Source Code

multiport_sram.v

```

1  `timescale 1ns / 1ps
2
3  module multiport_sram #(
4      parameter DEPTH = 256,
5      parameter DATA_W = 8
6  ) (
7      input wire clk,
8
9      // 1 Write Port (For the external bus)
10     input wire wr_en,
11     input wire [$clog2(DEPTH)-1:0] wr_addr,
12     input wire [DATA_W-1:0] wr_data,
13
14     // 3 Parallel Read Ports (For the Engine)
15     input wire [$clog2(DEPTH)-1:0] rd_addr0,
16     output reg [DATA_W-1:0] rd_data0,
17
18     input wire [$clog2(DEPTH)-1:0] rd_addr1,
19     output reg [DATA_W-1:0] rd_data1,
20
21     input wire [$clog2(DEPTH)-1:0] rd_addr2,
22     output reg [DATA_W-1:0] rd_data2
23 );
24
25     reg [DATA_W-1:0] mem_array [0:DEPTH-1];
26
27     always @(posedge clk) begin
28         if (wr_en) begin
29             mem_array[wr_addr] <= wr_data;
30         end
31
32         // Fetch 3 addresses simultaneously!
33         rd_data0 <= mem_array[rd_addr0];
34         rd_data1 <= mem_array[rd_addr1];
35         rd_data2 <= mem_array[rd_addr2];
36     end

```

```
37
38 endmodule
```

Listing 12: Source code for the multi-port SRAM with 3 parallel read ports

conv_engine_parallel.v

```
1 `timescale 1ns / 1ps
2
3 module conv_engine_parallel #(
4     parameter H      = 16,
5     parameter W      = 16,
6     parameter K      = 3,
7     parameter DATA_W = 8,
8     parameter ACC_W   = 32
9 ) (
10     input  wire clk,
11     input  wire rst,
12     input  wire start,
13
14     // 3 Parallel Input Read Ports
15     output reg  [$clog2(H*W)-1:0] in_rd_addr0, in_rd_addr1, in_rd_addr2,
16     input  wire signed [DATA_W-1:0] in_rd_data0, in_rd_data1, in_rd_data2,
17
18     // 3 Parallel Kernel Read Ports
19     output reg  [$clog2(K*K)-1:0] ker_rd_addr0, ker_rd_addr1, ker_rd_addr2,
20     input  wire signed [DATA_W-1:0] ker_rd_data0, ker_rd_data1, ker_rd_data2,
21
22     // Output Port
23     output reg                                output_wr_en,
24     output reg  [$clog2((H-K+1)*(W-K+1))-1:0] output_wr_addr,
25     output reg  signed [DATA_W-1:0]          output_wr_data,
26
27     output reg  done
28 );
29
30 localparam ST_WAIT_START = 3'd0,
31            ST_FETCH_ROW   = 3'd1,
32            ST_MAC_ROW     = 3'd2,
33            ST_STORE_PIXEL = 3'd3,
34            ST_DONE_CONV   = 3'd4;
35
36 reg [2:0] state;
37 reg [$clog2(H)-1:0] out_row;
38 reg [$clog2(W)-1:0] out_col;
39 reg [$clog2(K):0]   ki; // Notice: kj is GONE!
40 reg signed [ACC_W-1:0] acc;
41
42 always @(*) begin
43     // Ask for 3 consecutive pixels in a row
44     in_rd_addr0 = ((out_row + ki) << $clog2(W)) + out_col + 0;
45     in_rd_addr1 = ((out_row + ki) << $clog2(W)) + out_col + 1;
46     in_rd_addr2 = ((out_row + ki) << $clog2(W)) + out_col + 2;
47
48     // Ask for 3 consecutive kernel weights in a row
49     ker_rd_addr0 = (ki * K) + 0;
50     ker_rd_addr1 = (ki * K) + 1;
51     ker_rd_addr2 = (ki * K) + 2;
52
53     output_wr_addr = (out_row * (W - K + 1)) + out_col;
54     output_wr_en   = (state == ST_STORE_PIXEL);
55     output_wr_data = acc[7:0];
56 end
57
58 always @(posedge clk) begin
59     if (rst) begin
60         state <= ST_WAIT_START; done <= 0; out_row <= 0; out_col <= 0; ki <= 0; acc
61         <= 0;
62     end else begin
63         case (state)
64             ST_WAIT_START: begin
65                 done <= 0;
```



```

65         if (start) begin state <= ST_FETCH_ROW; out_row<=0; out_col<=0; ki
        <=0; acc<=0; end
66     end
67
68     ST_FETCH_ROW: state <= ST_MAC_ROW;
69
70     ST_MAC_ROW: begin
71         // THE EXTENSION: 3 Multipliers and an Adder Tree in 1 cycle!
72         acc <= acc + (in_rd_data0 * ker_rd_data0)
73                 + (in_rd_data1 * ker_rd_data1)
74                 + (in_rd_data2 * ker_rd_data2);
75
76         if (ki == K - 1) begin
77             state <= ST_STORE_PIXEL;
78         end else begin
79             ki <= ki + 1;
80             state <= ST_FETCH_ROW;
81         end
82     end
83
84     ST_STORE_PIXEL: begin
85         acc <= 0; ki <= 0;
86         if (out_col == (W - K)) begin
87             out_col <= 0;
88             if (out_row == (H - K)) state <= ST_DONE_CONV;
89             else begin out_row <= out_row + 1; state <= ST_FETCH_ROW; end
90         end else begin out_col <= out_col + 1; state <= ST_FETCH_ROW; end
91     end
92
93     ST_DONE_CONV: begin done <= 1; if (!start) state <= ST_WAIT_START; end
94 endcase
95 end
96 end
97 endmodule

```

Listing 13: Source code for the parallel convolution engine featuring a 3-input adder tree

conv2d_accel.v (Parallel Wrapper)

```

1  `timescale 1ns / 1ps
2
3  module conv2d_accel #(
4      parameter H      = 16,
5      parameter W      = 16,
6      parameter K      = 3,
7      parameter DATA_W = 8,
8      parameter ACC_W   = 32,
9      parameter ADDR_W  = 14
10 ) (
11     input  wire      clk,
12     input  wire      rst,
13
14     input  wire      wr_en,
15     input  wire [ADDR_W-1:0] wr_addr,
16     input  wire [DATA_W-1:0] wr_data,
17     output reg      wr_ready,
18
19     input  wire      rd_en,
20     input  wire [ADDR_W-1:0] rd_addr,
21     output reg [DATA_W-1:0] rd_data,
22     output reg      rd_valid
23 );
24
25 // 1. ENGINE SIGNALS & BUS DECODING
26 wire start, done;
27
28 wire      eng_out_wr_en;
29 wire [31:0] eng_out_wr_addr;
30 wire [DATA_W-1:0] eng_out_wr_data;
31
32 reg [31:0] cycle_count;
33 reg      counting;
34

```

```

35 wire [1:0] bank_sel_wr = wr_addr[13:12];
36 wire [1:0] bank_sel_rd = rd_addr[13:12];
37
38 wire ext_in_wr_en = wr_en & (bank_sel_wr == 2'b00);
39 wire ext_ker_wr_en = wr_en & (bank_sel_wr == 2'b01);
40 wire csr_write_start = wr_en & (bank_sel_wr == 2'b11) & (wr_addr[11:0] == 12'h000);
41 assign start = csr_write_start & (wr_data == 8'h01);
42
43 // 2. PARALLEL WIRES & INSTANTIATIONS
44
45 // 3 parallel wires for the Input Image
46 wire [$clog2(H*W)-1:0] in_a0, in_a1, in_a2;
47 wire [DATA_W-1:0] in_d0, in_d1, in_d2;
48
49 // 3 parallel wires for the Kernel
50 wire [$clog2(K*K)-1:0] ker_a0, ker_a1, ker_a2;
51 wire [DATA_W-1:0] ker_d0, ker_d1, ker_d2;
52
53 multiport_sram #(.DEPTH(H*W), .DATA_W(DATA_W)) input_sram (
54     .clk(clk), .wr_en(ext_in_wr_en), .wr_addr(wr_addr[$clog2(H*W)-1:0]), .wr_data(
55         wr_data),
56     .rd_addr0(in_a0), .rd_data0(in_d0),
57     .rd_addr1(in_a1), .rd_data1(in_d1),
58     .rd_addr2(in_a2), .rd_data2(in_d2)
59 );
60
61 multiport_sram #(.DEPTH(K*K), .DATA_W(DATA_W)) kernel_sram (
62     .clk(clk), .wr_en(ext_ker_wr_en), .wr_addr(wr_addr[$clog2(K*K)-1:0]), .wr_data(
63         wr_data),
64     .rd_addr0(ker_a0), .rd_data0(ker_d0),
65     .rd_addr1(ker_a1), .rd_data1(ker_d1),
66     .rd_addr2(ker_a2), .rd_data2(ker_d2)
67 );
68
69 wire [DATA_W-1:0] ext_out_rd_data;
70
71 simple_sram #(.DEPTH((H-K+1)*(W-K+1)), .DATA_W(DATA_W)) output_sram (
72     .clk(clk), .wr_en(eng_out_wr_en), .wr_addr(eng_out_wr_addr), .wr_data(
73         eng_out_wr_data),
74     .rd_addr(rd_addr[$clog2((H-K+1)*(W-K+1))-1:0]), .rd_data(ext_out_rd_data)
75 );
76
77 conv_engine_parallel #(.H(H), .W(W), .K(K), .DATA_W(DATA_W), .ACC_W(ACC_W)) engine (
78     .clk(clk), .rst(rst), .start(start),
79     .in_rd_addr0(in_a0), .in_rd_data0(in_d0),
80     .in_rd_addr1(in_a1), .in_rd_data1(in_d1),
81     .in_rd_addr2(in_a2), .in_rd_data2(in_d2),
82     .ker_rd_addr0(ker_a0), .ker_rd_data0(ker_d0),
83     .ker_rd_addr1(ker_a1), .ker_rd_data1(ker_d1),
84     .ker_rd_addr2(ker_a2), .ker_rd_data2(ker_d2),
85     .output_wr_en(eng_out_wr_en), .output_wr_addr(eng_out_wr_addr), .output_wr_data(
86         eng_out_wr_data),
87     .done(done)
88 );
89
90 // 3. BUS READOUT & CYCLE COUNTER
91 always @(*) begin
92     rd_data = 8'h00;
93     if (rd_en) begin
94         case (bank_sel_rd)
95             2'b10: rd_data = ext_out_rd_data;
96             2'b11: begin
97                 if (rd_addr[11:0] == 12'h004) rd_data = {7'b0, done};
98                 else if (rd_addr[11:0] == 12'h008) rd_data = cycle_count[7:0];
99                 else if (rd_addr[11:0] == 12'h009) rd_data = cycle_count[15:8];
100             end
101         endcase
102     end
103 end
104
105 always @(posedge clk) begin
106     if (rst) begin
107         wr_ready <= 0;

```

```

104         rd_valid    <= 0;
105         cycle_count <= 0;
106         counting    <= 0;
107     end else begin
108         wr_ready <= wr_en;
109         rd_valid <= rd_en;
110
111         if (start) begin
112             counting    <= 1;
113             cycle_count <= 0;
114         end else if (done) begin
115             counting    <= 0;
116         end else if (counting) begin
117             cycle_count <= cycle_count + 1;
118         end
119     end
120 end
121 endmodule

```

Listing 14: Source code for the modified top-level wrapper accommodating the multi-port SRAMs

6.4 Simulation Testbench

```

1 Starting test:          Python CNN Data (16x16)
2 Hardware finished in 1373 clock cycles
3 Verifying output pixels against expected logic
4 [PASS] Pixel 0: matched 8f
5 [PASS] Pixel 1: matched 06
6 [PASS] Pixel 2: matched 3b
7 [PASS] Pixel 3: matched 4d
8 [PASS] Pixel 4: matched 00
9 [PASS] Pixel 5: matched 00
10 [PASS] Pixel 6: matched 00
11 [PASS] Pixel 7: matched 00
12 [PASS] Pixel 8: matched 00
13 [PASS] Pixel 9: matched 00
14 [PASS] Pixel 10: matched db
15 [PASS] Pixel 11: matched 82
16 [PASS] Pixel 12: matched ab
17 [PASS] Pixel 13: matched 3c
18 [PASS] Pixel 14: matched cd
19 [PASS] Pixel 15: matched 4a
20 [PASS] Pixel 16: matched 35
21 [PASS] Pixel 17: matched 36
22 [PASS] Pixel 18: matched 00
23 [PASS] Pixel 19: matched 00
24 [PASS] Pixel 20: matched 00
25 [PASS] Pixel 21: matched 00
26 [PASS] Pixel 22: matched 00
27 [PASS] Pixel 23: matched 00
28 [PASS] Pixel 24: matched 85
29 [PASS] Pixel 25: matched fe
30 [PASS] Pixel 26: matched 4d
31 [PASS] Pixel 27: matched b4
32 [PASS] Pixel 28: matched c8
33 [PASS] Pixel 29: matched 71
34 [PASS] Pixel 30: matched 7b
35 [PASS] Pixel 31: matched cd
36 [PASS] Pixel 32: matched 00
37 [PASS] Pixel 33: matched 00
38 [PASS] Pixel 34: matched 00
39 [PASS] Pixel 35: matched 00
40 [PASS] Pixel 36: matched 00
41 [PASS] Pixel 37: matched 08
42 [PASS] Pixel 38: matched c6
43 [PASS] Pixel 39: matched 23
44 [PASS] Pixel 40: matched 5f
45 [PASS] Pixel 41: matched 45
46 [PASS] Pixel 42: matched b1
47 [PASS] Pixel 43: matched 65
48 [PASS] Pixel 44: matched 95
49 [PASS] Pixel 45: matched 7f

```

```

50 [PASS] Pixel 46: matched 00
51 [PASS] Pixel 47: matched 00
52 [PASS] Pixel 48: matched 00
53 [PASS] Pixel 49: matched 00
54 [PASS] Pixel 50: matched 00
55 [PASS] Pixel 51: matched 27
56 [PASS] Pixel 52: matched 79
57 [PASS] Pixel 53: matched 8f
58 [PASS] Pixel 54: matched 6f
59 [PASS] Pixel 55: matched 52
60 [PASS] Pixel 56: matched 89
61 [PASS] Pixel 57: matched 13
62 [PASS] Pixel 58: matched 34
63 [PASS] Pixel 59: matched 71
64 [PASS] Pixel 60: matched 04
65 [PASS] Pixel 61: matched 00
66 [PASS] Pixel 62: matched 00
67 [PASS] Pixel 63: matched 00
68 [PASS] Pixel 64: matched 00
69 [PASS] Pixel 65: matched 90
70 [PASS] Pixel 66: matched db
71 [PASS] Pixel 67: matched ad
72 [PASS] Pixel 68: matched 0e
73 [PASS] Pixel 69: matched b1
74 [PASS] Pixel 70: matched 6f
75 [PASS] Pixel 71: matched 78
76 [PASS] Pixel 72: matched 35
77 [PASS] Pixel 73: matched f0
78 [PASS] Pixel 74: matched 16
79 [PASS] Pixel 75: matched 41
80 [PASS] Pixel 76: matched 00
81 [PASS] Pixel 77: matched 00
82 [PASS] Pixel 78: matched 00
83 [PASS] Pixel 79: matched 50
84 [PASS] Pixel 80: matched 94
85 [PASS] Pixel 81: matched cc
86 [PASS] Pixel 82: matched 1c
87 [PASS] Pixel 83: matched 94
88 [PASS] Pixel 84: matched a1
89 [PASS] Pixel 85: matched 0e
90 [PASS] Pixel 86: matched 1a
91 [PASS] Pixel 87: matched e8
92 [PASS] Pixel 88: matched ea
93 [PASS] Pixel 89: matched 11
94 [PASS] Pixel 90: matched 75
95 [PASS] Pixel 91: matched 59
96 [PASS] Pixel 92: matched cd
97 [PASS] Pixel 93: matched ec
98 [PASS] Pixel 94: matched ff
99 [PASS] Pixel 95: matched 9a
100 [PASS] Pixel 96: matched bc
101 [PASS] Pixel 97: matched 64
102 [PASS] Pixel 98: matched 79
103 [PASS] Pixel 99: matched ef
104 [PASS] Pixel 100: matched e3
105 [PASS] Pixel 101: matched 51
106 [PASS] Pixel 102: matched ad
107 [PASS] Pixel 103: matched d3
108 [PASS] Pixel 104: matched 5f
109 [PASS] Pixel 105: matched 3f
110 [PASS] Pixel 106: matched 5e
111 [PASS] Pixel 107: matched 84
112 [PASS] Pixel 108: matched 81
113 [PASS] Pixel 109: matched 08
114 [PASS] Pixel 110: matched 81
115 [PASS] Pixel 111: matched c6
116 [PASS] Pixel 112: matched dd
117 [PASS] Pixel 113: matched 16
118 [PASS] Pixel 114: matched e6
119 [PASS] Pixel 115: matched 29
120 [PASS] Pixel 116: matched e0
121 [PASS] Pixel 117: matched 77
122 [PASS] Pixel 118: matched 5f

```

```

123 [PASS] Pixel 119: matched a9
124 [PASS] Pixel 120: matched 53
125 [PASS] Pixel 121: matched bd
126 [PASS] Pixel 122: matched af
127 [PASS] Pixel 123: matched c1
128 [PASS] Pixel 124: matched 00
129 [PASS] Pixel 125: matched 0c
130 [PASS] Pixel 126: matched 00
131 [PASS] Pixel 127: matched 58
132 [PASS] Pixel 128: matched 24
133 [PASS] Pixel 129: matched 1d
134 [PASS] Pixel 130: matched 0b
135 [PASS] Pixel 131: matched 74
136 [PASS] Pixel 132: matched 75
137 [PASS] Pixel 133: matched f9
138 [PASS] Pixel 134: matched 24
139 [PASS] Pixel 135: matched 62
140 [PASS] Pixel 136: matched e8
141 [PASS] Pixel 137: matched 6b
142 [PASS] Pixel 138: matched 88
143 [PASS] Pixel 139: matched 89
144 [PASS] Pixel 140: matched 00
145 [PASS] Pixel 141: matched 00
146 [PASS] Pixel 142: matched 00
147 [PASS] Pixel 143: matched 00
148 [PASS] Pixel 144: matched 00
149 [PASS] Pixel 145: matched 00
150 [PASS] Pixel 146: matched 00
151 [PASS] Pixel 147: matched 36
152 [PASS] Pixel 148: matched 2a
153 [PASS] Pixel 149: matched a6
154 [PASS] Pixel 150: matched 21
155 [PASS] Pixel 151: matched 13
156 [PASS] Pixel 152: matched cd
157 [PASS] Pixel 153: matched 29
158 [PASS] Pixel 154: matched 00
159 [PASS] Pixel 155: matched 00
160 [PASS] Pixel 156: matched 00
161 [PASS] Pixel 157: matched 00
162 [PASS] Pixel 158: matched 00
163 [PASS] Pixel 159: matched 00
164 [PASS] Pixel 160: matched 00
165 [PASS] Pixel 161: matched 00
166 [PASS] Pixel 162: matched d8
167 [PASS] Pixel 163: matched a4
168 [PASS] Pixel 164: matched 62
169 [PASS] Pixel 165: matched 38
170 [PASS] Pixel 166: matched a2
171 [PASS] Pixel 167: matched 00
172 [PASS] Pixel 168: matched 00
173 [PASS] Pixel 169: matched 00
174 [PASS] Pixel 170: matched 00
175 [PASS] Pixel 171: matched 00
176 [PASS] Pixel 172: matched 00
177 [PASS] Pixel 173: matched 00
178 [PASS] Pixel 174: matched 00
179 [PASS] Pixel 175: matched 00
180 [PASS] Pixel 176: matched 8e
181 [PASS] Pixel 177: matched 10
182 [PASS] Pixel 178: matched d6
183 [PASS] Pixel 179: matched b4
184 [PASS] Pixel 180: matched 60
185 [PASS] Pixel 181: matched 00
186 [PASS] Pixel 182: matched 00
187 [PASS] Pixel 183: matched 00
188 [PASS] Pixel 184: matched 00
189 [PASS] Pixel 185: matched 00
190 [PASS] Pixel 186: matched 00
191 [PASS] Pixel 187: matched 00
192 [PASS] Pixel 188: matched 00
193 [PASS] Pixel 189: matched 00
194 [PASS] Pixel 190: matched 22
195 [PASS] Pixel 191: matched 38

```

```

196 [PASS] Pixel 192: matched 6a
197 [PASS] Pixel 193: matched b4
198 [PASS] Pixel 194: matched 60
199 [PASS] Pixel 195: matched 00
200 Test passed with no errors
201
202 Sign-off approved: Hardware verification complete with 0 errors

```

Listing 15: Testbench Console Output (Randomized Data Simulation) for the extension part



Figure 10: Testbench Console Output showing PASS result

6.5 Waveform Analysis

To verify the functional correctness and timing behavior of the partially parallel architecture, we analyzed the simulation waveforms. While the external interface protocol remains identical to the baseline design, the internal execution speed demonstrates a massive improvement.

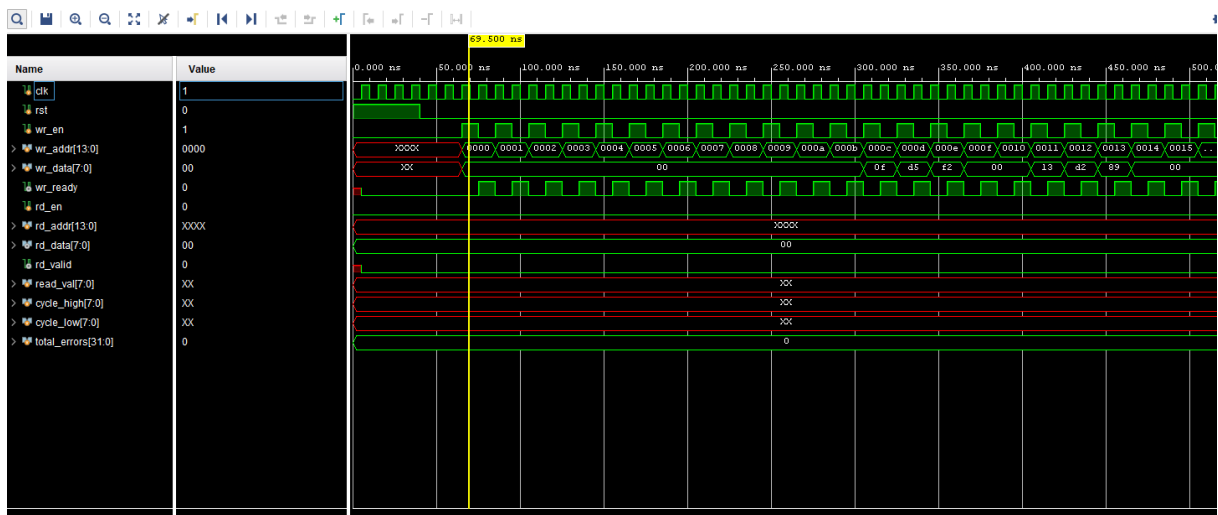


Figure 11: Parallel Extension - Phase 1: Serial Data Loading

As depicted in Figure 11, the Data Loading phase remains functionally identical to the baseline serial architecture. Since the external system bus is 8-bit wide, the input feature map and kernel weights must still be written sequentially into the multi-port SRAMs. The write enable signal (`wr_en`) stays asserted while the address increments.



Figure 12: Parallel Extension - Phase 2: Accelerated Computation and Polling

Figure 12 highlights the primary advantage of the extension. After the testbench asserts the START command by writing 01 to the CSR at 14'h3000, the parallel engine activates. Because the datapath processes three pixels simultaneously, the overall computation duration is visibly and drastically shorter. The polling phase concludes quickly, with the cycle counters (`cycle_high` and `cycle_low`) freezing at a combined value of 1373 (decimal) clock cycles—a massive reduction from the original 3726 cycles.

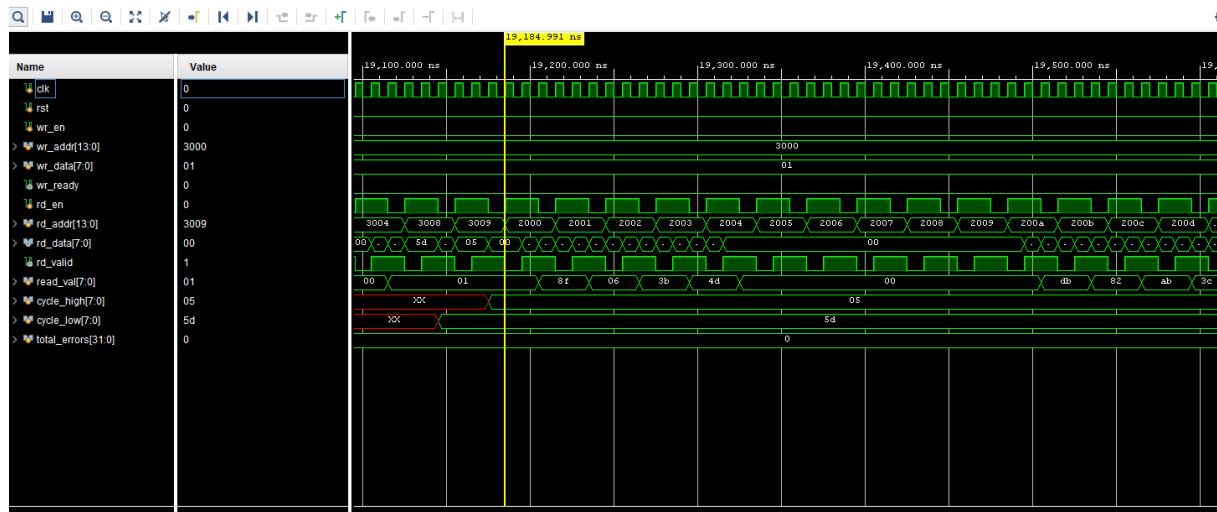


Figure 13: Parallel Extension - Phase 3: Result Readback Verification

Finally, Figure 13 confirms the architectural integrity of our parallel implementation. During the Result Readback phase, the testbench reads the output SRAM starting from address 14'h2000. Despite the complex 3-multiplier adder tree and parallel fetching logic, the `total_errors` signal strictly remains at 0. This indisputably proves that the parallel hardware successfully achieves high-throughput acceleration without sacrificing any computational accuracy.

6.6 Synthesis Results

Metric	Value
Input size ($H \times W$)	16×16
Kernel size ($K \times K$)	3×3
Output size	14×14
Total MAC operations	1764
LUTs	423
LUTRAMs	184
FFs	98
DSPs	0
BRAMs	0
WNS (ns)	7.424
Max frequency (MHz)	388.20
Clock cycles for computation	1373
Time for one convolution (μs)	13.73
Throughput (MAC operations / second)	128.48 Million

Table 3: Extension: Partially Parallel Accelerator Performance Results

Summary

Resource	Utilization	Available	Utilization %
LUT	423	230400	0.18
LUTRAM	184	101760	0.18
FF	98	460800	0.02
IO	50	360	13.89
BUFG	1	544	0.18

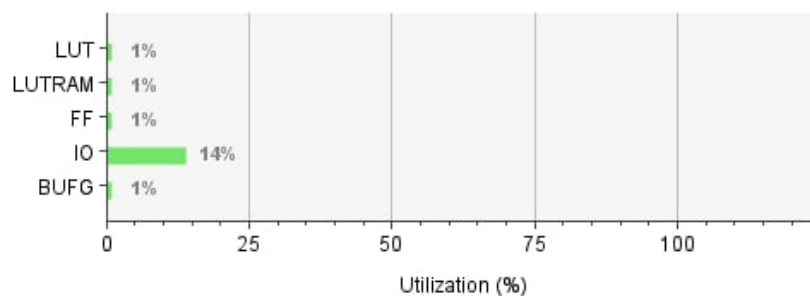


Figure 14: Vivado Utilization Report confirming the use of 423 LUTs, 184 LUTRAMs, and 98 FFs.

Design Timing Summary

Setup	Hold	Pulse Width
Worst Negative Slack (WNS): 7.424 ns	Worst Hold Slack (WHS): -0.076 ns	Worst Pulse Width Slack (WPWS): 4.468 ns
Total Negative Slack (TNS): 0.000 ns	Total Hold Slack (THS): -6.061 ns	Total Pulse Width Negative Slack (TPWS): 0.000 ns
Number of Failing Endpoints: 0	Number of Failing Endpoints: 156	Number of Failing Endpoints: 0
Total Number of Endpoints: 450	Total Number of Endpoints: 450	Total Number of Endpoints: 307

Timing constraints are not met.

Figure 15: Vivado Timing Summary confirming a Worst Negative Slack (WNS) of 7.424 ns.

6.7 Architectural Trade-off Analysis

Comparing these results with the baseline serial design reveals the classic hardware trade-off between Area and Speed:

- **Speedup:** The execution cycles drastically dropped from 3726 to 1373. This is an effective speedup of $\approx 2.71\times$, which closely aligns with the theoretical maximum speedup of $3\times$. The minor discrepancy is due to the inherent FSM overhead states (`ST_FETCH_ROW` and `ST_STORE_PIXEL`) taking additional cycles. The overall throughput surged to over 128 MMAC/s.
- **Area Penalty:** The logic footprint expanded significantly. LUT utilization nearly doubled from 218 to 423 ($\approx 1.9\times$ increase). This is driven by two factors: the instantiation of the 3-input adder tree, and the memory architecture. Because physical Xilinx BRAM primitives are strictly dual-port, our custom 4-port SRAM (1 write, 3 read) forced Vivado to synthesize the memory entirely out of Distributed RAM, evidenced by the 184 LUTRAMs consumed and 0 BRAMs utilized.
- **Timing Constraints:** The Worst Negative Slack (WNS) slightly degraded from 7.926 ns to 7.424 ns, reducing the maximum operating frequency from 482.16 MHz to 388.20 MHz. This is a direct consequence of the deepened combinational logic path caused by stacking three multipliers and two adders before the accumulator register.

Ultimately, the extension proves to be highly successful, demonstrating that parallelization can significantly boost throughput for CNN edge-computing applications at a very reasonable cost in FPGA logic resources.

7 Team Contributions

The success of this project was a highly collaborative effort. The specific responsibilities and contributions of each team member are detailed in the table below:

Name	Role	Contr.	Specific Contributions
Nguyen Huy Long	Project Lead, Hardware	100%	• Architected the core 2D convolution datapath and FSM. • Implemented key Verilog modules (<code>conv_engine</code>, <code>simple_sram</code>). • Developed advanced verification test-bench. • Performed RTL synthesis and timing closure on Vivado. • Authored, structured, and formatted the final technical report.
Pham Trung Kien	Hardware Engineer	100%	• Assisted in RTL coding and system-level module integration. • Conducted behavioral simulations and debugged memory addressing issues. • Analyzed Vivado utilization reports and performance metrics.
Bui Tuan Kien	Software Engineer	100%	• Researched and developed the Python-based CNN software model. • Handled data preparation (MNIST cropping, <code>int8</code> quantization). • Generated the golden reference hex files for hardware verification.
Phung Minh Khoa	Extension Developer	100%	• Researched and implemented the project extension (Partially Parallel Architecture). • Documented the system architecture and extension results in the final report.

Table 4: Team Member Roles and Contributions